

FORMATO DE ENTREGA DE EVIDENCIAS

Serie de guías · Desarrollo de APIs con FastAPI

1. Datos del aprendiz

Nombre completo	Joel Andrés Mendoza Buriticá
Número de documento	1042853355
Programa	Análisis y Desarrollo de Software (ADSO)
Ficha	3169892
Número de la guía	GA5 – Guía 5
Nombre de la guía	Persistencia SQLite
Instructor	Mateo Arroyave
Fecha de entrega	13 / 08 / 2026

Cómo diligenciar este formato

Complete cada evidencia reemplazando el texto en gris. Escriba dentro de los recuadros y pegue las capturas donde se indica. Al terminar, exporte a PDF (Archivo ▶ Guardar como ▶ PDF) y suba el archivo a Google Classroom en la actividad correspondiente.

2. Enlace al repositorio de GitHub

Muchas evidencias piden el código en GitHub. Pegue aquí el enlace de su repositorio (debe ser público):

<https://github.com/JBUR10/APIS-FASTAPIS>

3. Evidencia 1 — Reflexión inicial

Código de la evidencia	GA5-AA1-EV01
Tipo	Conocimiento

Respuestas a las preguntas de reflexión:

1. Antes (Creados sin recargar)

```
{
  "id": 4,
  "nombre": "papa",
  "precio": 2000,
  "categoria": "comida",
  "proveedor": "yo"
},
{
  "id": 5,
  "nombre": "balon",
  "precio": 20000,
  "categoria": "deportes",
  "proveedor": "tu"
},
{
  "id": 6,
  "nombre": "media",
  "precio": 5000,
  "categoria": "ropa",
  "proveedor": "él"
}
]
```

2. Después

Code Details

200

Response body

```
{
  {
    "id": 1,
    "nombre": "Teclado mecanico",
    "precio": 120000,
    "categoria": "Periféricos",
    "proveedor": "Manzana (Apol)"
  },
  {
    "id": 2,
    "nombre": "Mouse gamer",
    "precio": 85000,
    "categoria": "Periféricos",
    "proveedor": "Flemingo"
  },
  {
    "id": 3,
    "nombre": "Monitor 24",
    "precio": 650000,
    "categoria": "Pantallas",
    "proveedor": "Distribuciones Metrio"
  }
}
```

Download

3. ¿Qué pasó con los productos que había creado y por qué?

Desaparecieron ya que no existe una base de datos donde estos datos creados puedan ser guardados.

- Imagine que esa API es la de un banco, una EPS o una tienda que vende de verdad. ¿Qué consecuencias tendría este comportamiento para el negocio y para los usuarios?

Para el negocio sería imposible llevar una cuenta de lo gastado, lo que se debe surtir, lo cobrado ese día, etc., esto ya que los datos de un día se borrarían al cerrar la aplicación, para los usuarios podría generar inconvenientes si en esa tienda se “fía”, en una EPS sería mas grave aun ya que no sería posible un registro de pacientes, historias clínicas, etc.

- Mencione tres aplicaciones que usa a diario y explique qué información necesitan recordar entre una sesión y otra.

WhatsApp: necesita recordar mis conversaciones, contactos, grupos y archivos enviados o recibidos para que pueda acceder a ellos cuando vuelva a abrir la aplicación.

Instagram: necesita recordar mi cuenta, publicaciones que he visto o con las que he interactuado, personas que sigo y mis mensajes directos.

Spotify: necesita recordar mis listas de reproducción, canciones favoritas, historial de reproducción y preferencias musicales.

- ¿Por qué cree que no basta con guardar los datos en un archivo de texto plano o en un .json? Proponga al menos dos razones.

Porque si estos datos se guardan en texto plano, si son filtrados estos pueden ser vistos sin complicaciones por el filtrador, lo que pondría en riesgo al usuario y dejaría saber que la pagina es vulnerable. Además, daría a esta persona acceso a funciones y permisos que no debería tener, lo que podría llevar a la falla de la app.

4. Evidencia 2 — Contextualización

Código de la evidencia	Ej.: GA5-AA1-EV02
Tipo	Taller

1.

```
import sqlite3

conexion = sqlite3.connect("taller.db")
cursor = conexion.cursor()

cursor.execute("""
    CREATE TABLE IF NOT EXISTS estudiantes (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        nombre TEXT NOT NULL,
        edad INTEGER,
        promedio REAL
    )
""")

conexion.commit()
```

2.

```
cursor.execute(
    "INSERT INTO estudiantes (nombre, edad, promedio) VALUES (?, ?, ?)",
    ("Carlos", 20, 4.2),
)

estudiantes = [
    ("Joel", 22, 4.8),
    ("Luis", 19, 3.9),
    ("Sergio", 25, 4.5),
    ("Durman", 21, 3.7),
    ("Sebastian", 23, 4.9),
]

cursor.executemany(
    "INSERT INTO estudiantes (nombre, edad, promedio) VALUES (?, ?, ?)",
    estudiantes,
)

conexion.commit()
```

3.

```
cursor.execute("SELECT * FROM estudiantes")
filas = cursor.fetchall()

print("\n--- TODOS LOS ESTUDIANTES ---")

for fila in filas:
    print(fila)

cursor.execute("SELECT * FROM estudiantes WHERE edad > 20")
filas = cursor.fetchall()

print("\n--- ESTUDIANTES MAYORES DE 20 ---")

for fila in filas:
    print(fila)

cursor.execute("SELECT * FROM estudiantes ORDER BY promedio DESC")
filas = cursor.fetchall()

print("\n--- ORDENADOS POR PROMEDIO ---")

for fila in filas:
    print(fila)

cursor.execute("SELECT * FROM estudiantes ORDER BY promedio DESC LIMIT 3")
filas = cursor.fetchall()

print("\n--- LOS 3 MEJORES PROMEDIOS ---")

for fila in filas:
    print(fila)

cursor.execute("SELECT * FROM estudiantes")
fila = cursor.fetchone()

print("\n--- FETCHONE ---")
print(fila)
```

```

--- TODOS LOS ESTUDIANTES ---
(1, 'Carlos', 20, 4.2)
(2, 'Joel', 22, 4.8)
(3, 'Luis', 19, 3.9)
(4, 'Sergio', 25, 4.5)
(5, 'Durman', 21, 3.7)
(6, 'Sebastian', 23, 4.9)

--- ESTUDIANTES MAYORES DE 20 ---
(2, 'Joel', 22, 4.8)
(4, 'Sergio', 25, 4.5)
(5, 'Durman', 21, 3.7)
(6, 'Sebastian', 23, 4.9)

--- ORDENADOS POR PROMEDIO ---
(6, 'Sebastian', 23, 4.9)
(2, 'Joel', 22, 4.8)
(4, 'Sergio', 25, 4.5)
(1, 'Carlos', 20, 4.2)
(3, 'Luis', 19, 3.9)
(5, 'Durman', 21, 3.7)

--- LOS 3 MEJORES PROMEDIOS ---
(6, 'Sebastian', 23, 4.9)
(2, 'Joel', 22, 4.8)
(4, 'Sergio', 25, 4.5)

--- FETCHONE ---
(1, 'Carlos', 20, 4.2)

```

4.

```

cursor.execute(
    "UPDATE estudiantes SET promedio = ? WHERE id = ?",
    (4.7, 1),
)

print("\nFilas actualizadas:", cursor.rowcount)

conexion.commit()

cursor.execute(
    "DELETE FROM estudiantes WHERE id = ?",
    (2,),
)

print("Filas eliminadas:", cursor.rowcount)

conexion.commit()

conexion.row_factory = sqlite3.Row
cursor = conexion.cursor()

cursor.execute("SELECT * FROM estudiantes")
filas = cursor.fetchall()

```

```
Filas actualizadas: 1
Filas eliminadas: 1
```

5.

```
print("\n--- USANDO sqlite3.Row ---")

for fila in filas:
    print("Nombre:", fila["nombre"])
    print("Edad:", fila["edad"])
    print("Promedio:", fila["promedio"])
    print("Como diccionario:", dict(fila))
    print()
```

```
--- USANDO sqlite3.Row ---
Nombre: Carlos
Edad: 20
Promedio: 4.7
Como diccionario: {'id': 1, 'nombre': 'Carlos', 'edad': 20, 'promedio': 4.7}

Nombre: Luis
Edad: 19
Promedio: 3.9
Como diccionario: {'id': 3, 'nombre': 'Luis', 'edad': 19, 'promedio': 3.9}

Nombre: Sergio
Edad: 25
Promedio: 4.5
Como diccionario: {'id': 4, 'nombre': 'Sergio', 'edad': 25, 'promedio': 4.5}

Nombre: Durman
Edad: 21
Promedio: 3.7
Como diccionario: {'id': 5, 'nombre': 'Durman', 'edad': 21, 'promedio': 3.7}

Nombre: Sebastian
Edad: 23
Promedio: 4.9
Como diccionario: {'id': 6, 'nombre': 'Sebastian', 'edad': 23, 'promedio': 4.9}
```

Paso 4:

```
consulta_insegura = f"""
    SELECT * FROM estudiantes
    WHERE nombre = '{dato}'
    """
```

```
SELECT * FROM estudiantes
WHERE nombre = '' OR '1'='1'
```

```
{'id': 1, 'nombre': 'Carlos', 'edad': 20, 'promedio': 4.7}
{'id': 3, 'nombre': 'Luis', 'edad': 19, 'promedio': 3.9}
{'id': 4, 'nombre': 'Sergio', 'edad': 25, 'promedio': 4.5}
{'id': 5, 'nombre': 'Durman', 'edad': 21, 'promedio': 3.7}
{'id': 6, 'nombre': 'Sebastian', 'edad': 23, 'promedio': 4.9}
{'id': 7, 'nombre': 'Carlos', 'edad': 20, 'promedio': 4.2}
{'id': 8, 'nombre': 'Joel', 'edad': 22, 'promedio': 4.8}
{'id': 9, 'nombre': 'Luis', 'edad': 19, 'promedio': 3.9}
{'id': 10, 'nombre': 'Sergio', 'edad': 25, 'promedio': 4.5}
{'id': 11, 'nombre': 'Durman', 'edad': 21, 'promedio': 3.7}
{'id': 12, 'nombre': 'Sebastian', 'edad': 23, 'promedio': 4.9}
{'id': 13, 'nombre': 'Carlos', 'edad': 20, 'promedio': 4.2}
{'id': 14, 'nombre': 'Joel', 'edad': 22, 'promedio': 4.8}
{'id': 15, 'nombre': 'Luis', 'edad': 19, 'promedio': 3.9}
{'id': 16, 'nombre': 'Sergio', 'edad': 25, 'promedio': 4.5}
{'id': 17, 'nombre': 'Durman', 'edad': 21, 'promedio': 3.7}
{'id': 18, 'nombre': 'Sebastian', 'edad': 23, 'promedio': 4.9}
```

```
consulta_segura = ""
```

```
SELECT * FROM estudiantes
```

```
WHERE nombre = ?
```

```
"""
```

```
Resultado:
```

```
Conexión cerrada.
```

```
(venv)
```

```
C:\Users\user>python main.py
```

5. Evidencia 3 — Apropiación (laboratorio)

Código de la evidencia	GA5-AA3-EV03
Tipo	Práctica guiada

Descripción de lo que hizo: explique brevemente los pasos que siguió y los ejercicios que resolvió.

Paso 2: Realizo la creación de las tablas de las bases de datos al momento en el que se ejecuta el main usando el uunicorn, estas tablas solo se crean si no existen ya en la base de datos.

```
def obtener_conexion():
    conexion = sqlite3.connect(DB_NAME, check_same_thread=False)
    conexion.row_factory = sqlite3.Row
    conexion.execute("PRAGMA foreign_keys = ON")
    return conexion

def crearTablas():
    conexion = obtener_conexion()
    cursor = conexion.cursor()

    cursor.execute("""
        CREATE TABLE IF NOT EXISTS Categoria (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            Nombre TEXT NOT NULL UNIQUE
        )
    """)
```

```
cursor.execute("""
    CREATE TABLE IF NOT EXISTS categorias (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        nombre TEXT NOT NULL UNIQUE
    )
""")

cursor.execute("""
    CREATE TABLE IF NOT EXISTS productos (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        nombre TEXT NOT NULL,
        precio REAL,
        categoria_id INTEGER,
        FOREIGN KEY (categoria_id) REFERENCES categorias(id)
    )
""")

cursor.execute("""
    CREATE TABLE IF NOT EXISTS usuarios (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        username TEXT NOT NULL UNIQUE,
        nombre TEXT NOT NULL,
        password TEXT NOT NULL,
        rol TEXT NOT NULL
    )
""")
```


Paso 3: Uso lifespan para la ejecución del código al iniciar la aplicación, aquí llamo a la función crear tablas y sembrar datos (que es la que se encarga de añadir los datos iniciales que ya hacíamos anteriormente) todo esto desde el archivo db donde se crean las tablas. Al funcionar correctamente se crea el archivo tienda.db con las tablas creadas.

```
from database import db
```

```
@asynccontextmanager
```

```
async def lifespan(app: FastAPI):
```

```
    db.crear_tablas()
```

```
    db.sembrar_datos()
```

```
    yield
```

```
JOELBUR03, 2 days ago • feat: Actualizar tabla productos a u  
app = FastAPI(title="API de la Tienda", lifespan=lifespan)
```

```
app.include_router(auth.router)
```

```
app.include_router(productos.router)
```

```
app.include_router(categorias.router)
```

```
app.include_router(proveedores.router)
```

```
app.include_router(empleados.router)
```

```
≡ store.db
```

```
U
```

Paso 4: Se reescribe productos.py para que sea compatible con SQLite sin problemas, aunque la guía no lo pide ya que en anteriores guías se pidió añadir más tablas tengo una conexión extra con el proveedor.

Adjunto solo una pequeña parte ya que el archivo tiene bastante información:

```

@router.get("")
def listar_productos():
    conexion = db.obtener_conexion()

    cursor = conexion.cursor()
    cursor.execute("""
        SELECT
            p.id,
            p.nombre,
            p.precio,
            p.id_categoria,
            c.nombre AS categoria,
            p.id_proveedor,
            pr.nombre AS proveedor
        FROM Producto p
        JOIN Categoria c
            ON p.id_categoria = c.id
        JOIN Proveedor pr
            ON p.id_proveedor = pr.id
    """)

    filas = cursor.fetchall()

    conexion.close()

    return [dict(fila) for fila in filas]

```

Persistencia: Hago las pruebas pertinentes para comprobar que los nuevos productos SI se guardan en la base de datos

Request URL

http://127.0.0.1:8000/productos/1

Server response

Code	Details
------	---------

Details

200

Response body

```
{
  "id": 1,
  "nombre": "Teclado mecanico",
  "precio": 120000,
  "id_categoria": 1,
  "categoria": "Perifericos",
  "id_proveedor": 1,
  "proveedor": "Manzana (Apol)"
}
```

Response headers

```
content-length: 143
content-type: application/json
date: Thu, 13 Aug 2026 16:56:28 GMT
server: uvicorn
```

TABLES	id	nombre	precio	id_cat...	id_pr...
> Categoría					
> Producto					
> Proveedor					
	1	Teclado mecanico	120000		1

Prueba de permisos: Intentar hacer algo que solo un admin podría con un usuario normal

Request URL

<http://127.0.0.1:8000/productos/1>

Server response

Code

Details

403

Undocumented

Error: Forbidden

Response body

```
{
  "detail": "Requiere rol de administrador"
}
```

Tablas creadas:

TABLES
> Categoria
> Producto
> Proveedor
> sqlite_sequence
> Usuario

Contenido tablas:

id	nombre
1	Perifericos
2	Pantallas
3	Audio

id	nombre	precio	id_cat...	id_pr...
1	Teclado mecanico	120000	1	

id	nombre	telefono	correo
1	Manzana (Apol)	3001234567	contacto@apol.com
2	Distribuciones Metrio	3109876543	ventas@metrio.com
3	Flamingo	3204567890	info@flamingo.com

1	admin	Administrador	\$2b\$12\$xY8.6.dKKH92ayASYnsp
2	joel	Joel Cliente	\$2b\$12\$Rh4lWNTtShTnB0wmq1

6. Evidencia 4 — Transferencia (reto)

Código de la evidencia	GA5-AA4-EV04
Tipo	Reto

Enlace al repositorio con el reto resuelto:

<https://github.com/JBUR10/APIS-FASTAPIS>

Explique brevemente su solución:

¿Qué endpoints implementó? ¿Qué le costó más? ¿Cómo lo probó?

Se actualizo completamente los endpoints de categoría para funcionar con las consultas de la misma forma que lo hace productos, en base es bastante fácil de comprender la complicación puede ser entender en primera los JOIN de las llaves foráneas. Lo probe de la misma forma que siempre, usando el /docs.

Curl

```
curl -X 'GET' \  
  'http://127.0.0.1:8000/categorias' \  
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/categorias
```

Server response

Code

Details

200

Response body

```
[  
  {  
    "id": 1,  
    "nombre": "Periféricos"  
  },  
  {  
    "id": 2,  
    "nombre": "Pantallas"  
  },  
  {  
    "id": 3,  
    "nombre": "Audio"  
  }  
]
```

Request URL

`http://127.0.0.1:8000/categorias`

Server response

Code

Details

201

Response body

```
{
  "mensaje": "Categoria creada",
  "categoria": {
    "id": 4,
    "nombre": "Pandequeso"
  },
  "creado_por": "admin"
}
```

	id	nombre
	id	nombre
	id	nombre
1	1	Perifericos
2	2	Pantallas
3	3	Audio
4	4	Pandequeso

200

Response body

```
{
  "id": 2,
  "nombre": "Pantallas"
}
```


200

Response body

```
{
  "mensaje": "Categoria actualizada",
  "categoria": {
    "id": 4,
    "nombre": "Pan"
  },
  "actualizado_por": "admin"
}
```

id	nombre
1	Perifericos
2	Pantallas
3	Audio
4	pan

200

Response body

```
{
  "mensaje": "Categoria eliminada",
  "categoria_id": 4,
  "eliminado_por": "admin"
}
```

id	nombre
1	Perifericos
2	Pantallas
3	Audio

Request URL

`http://127.0.0.1:8000/categorias/3/productos`

Server response

Code	Details
------	---------

200	
-----	--

Response body

```
{
  "id": 3,
  "nombre": "Audio",
  "productos": []
}
```

404	
-----	--

Undocumented

Error: Not Found

Response body

```
{
  "detail": "Categoria no encontrada"
}
```

422	
-----	--

Error: Unprocessable Content

Response body

```
{
  "detail": [
    {
      "type": "missing",
      "loc": [
        "body",
        "nombre"
      ],
      "msg": "Field required",
      "input": {}
    }
  ]
}
```

7. Lista de chequeo (autoevaluación)

Antes de subir a Classroom, verifique que cumplió con todo. Marque cada casilla:

- ✓ ☐ Diligencí mis datos (nombre, documento, ficha, número de guía).
- ✓ ☐ Respondí la Evidencia 1 — Reflexión inicial.
- ✓ ☐ Entregué la Evidencia 2 — Cuadro/mapa de contextualización.
- ✓ ☐ Entregué la Evidencia 3 — Laboratorio, con capturas de /docs.
- ✓ ☐ Entregué la Evidencia 4 — Reto, con enlace a GitHub y capturas.
- ✓ ☐ Mi repositorio de GitHub es público y el enlace funciona.
- ✓ ☐ Incluí capturas de un error 404 y un error 422 donde se pidió.
- ✓ ☐ Exporté el documento a PDF antes de subirlo.